

PROJET ST2AIM · DEEP LEARNING

Apprendre à une machine à voir

Classification d'images chats / chiens avec un CNN

Victor Haegeman — EFREI

1 Le dataset : 3000 photos, 2 classes

2000 images d'entraînement · **1000** de validation

Classes parfaitement équilibrées (50 % / 50 %)
→ le hasard donne 50 %, c'est notre référence.

Les photos n'ont pas toutes la même taille

Le réseau attend une entrée fixe → tout est redimensionné en $150 \times 150 \times 3$
(3 = les canaux rouge, vert, bleu)

8 images du jeu d'entraînement (redimensionnées en 150x150)



Découpage des données

Jeu	Images	Rôle
Entraînement (80 % du dossier train)	1600	le modèle apprend ses poids
Validation (dossier validation)	1000	surveiller l'apprentissage à chaque epoch
Test (20 % du dossier train)	400	note finale, jamais vu avant

2 Pourquoi un CNN et pas un réseau dense ?

Une image 150×150 en couleur

67 500

valeurs en entrée

→ une seule couche dense de 1000 neurones = 67,5 millions de paramètres

- ✗ **Trop de paramètres**
mémoire saturée, sur-apprentissage immédiat
- ✗ **L'information spatiale est perdue**
le réseau ne sait plus quels pixels étaient voisins
- ✗ **Rien n'est réutilisé**
un motif appris en haut à gauche doit être réappris ailleurs

La réponse du CNN : deux idées simples

Connectivité locale

Chaque neurone ne regarde qu'un petit carré de 3×3 pixels, pas l'image entière.

Partage des poids

Le même filtre glisse sur toute l'image : un motif appris une fois est reconnu partout.

3 L'architecture : 4 blocs de convolution

Couche	Sortie	Paramètres
Rescaling (1/255)	150 × 150 × 3	0
Conv2D 32 filtres 3×3 + ReLU	148 × 148 × 32	896
MaxPooling 2×2	74 × 74 × 32	0
Conv2D 64 filtres 3×3 + ReLU	72 × 72 × 64	18 496
MaxPooling 2×2	36 × 36 × 64	0
Conv2D 128 filtres 3×3 + ReLU	34 × 34 × 128	73 856
MaxPooling 2×2	17 × 17 × 128	0
Conv2D 128 filtres 3×3 + ReLU	15 × 15 × 128	147 584
MaxPooling 2×2	7 × 7 × 128	0
Flatten	6272	0
Dense 128 + ReLU	128	802 944
Dense 1 + sigmoïde	1	129

Le calcul à la main

$$\text{params} = M \times (K \times K \times C_{in} + 1)$$

ex. bloc 2 : $64 \times (3 \times 3 \times 32 + 1) = 18\,496$

M = nb de filtres, K = taille du noyau, +1 = le biais

1 043 905

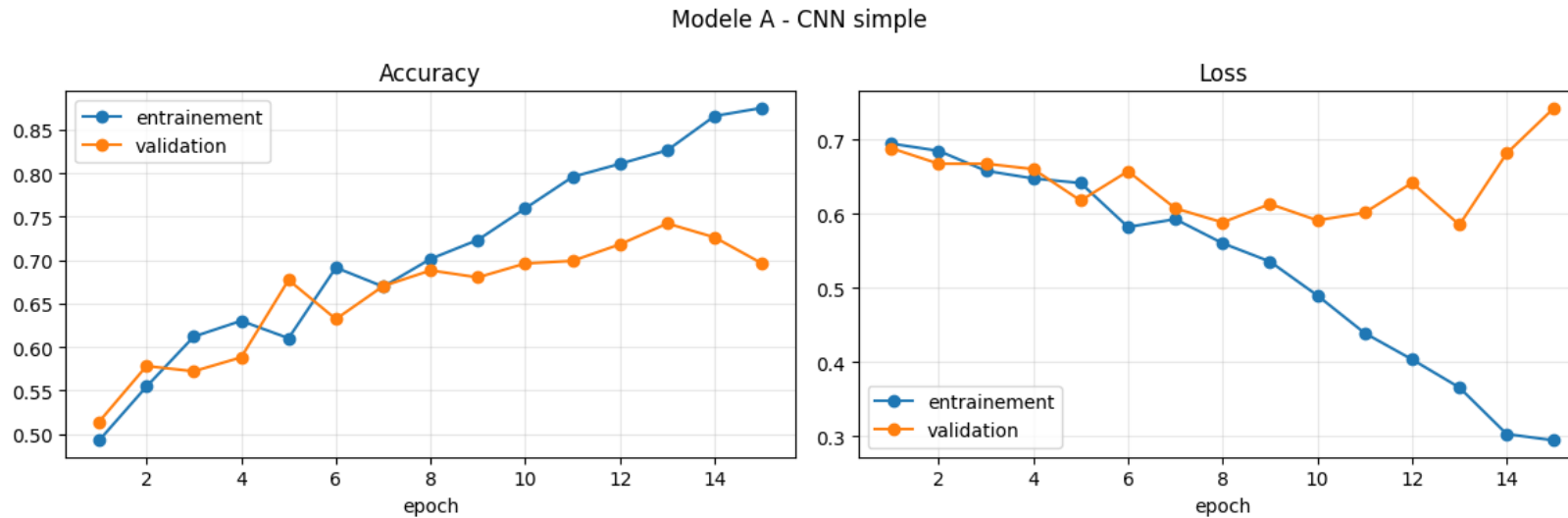
paramètres au total — chiffre retrouvé à la main puis vérifié avec `model.summary()`

Le point à retenir

77 % des paramètres sont dans la seule couche dense (802 944).

Les 4 blocs de convolution n'en utilisent que 240 832 : le partage des poids coûte très peu.

4 Entraînement : le modèle sur-apprend



Après 15 epochs

0,875

accuracy entraînement

0,696

accuracy validation

écart : 0,179

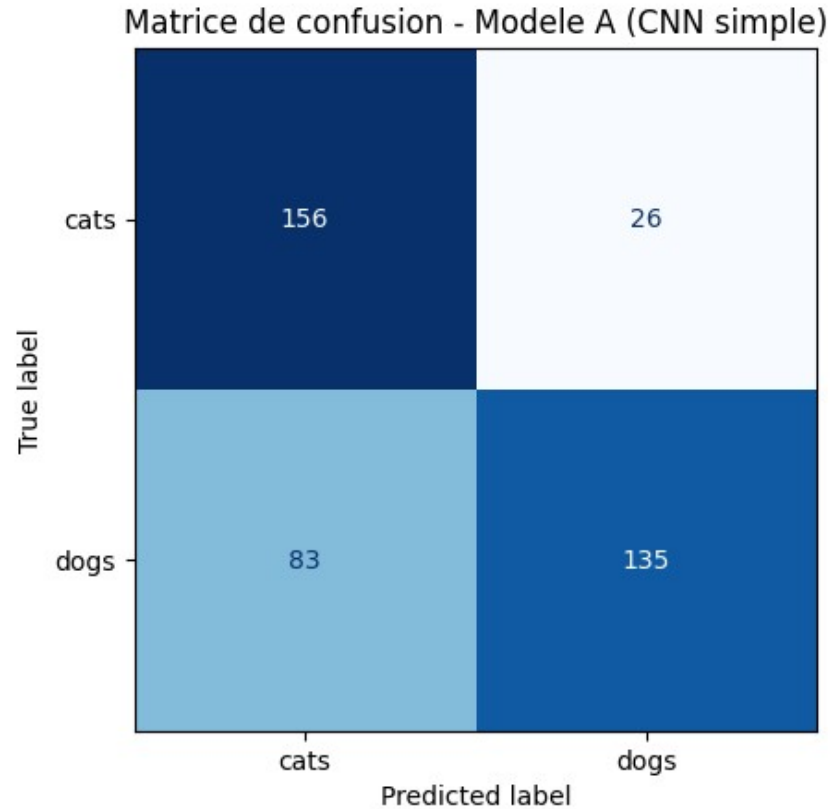
Le diagnostic : sur-apprentissage

La courbe d'entraînement monte toujours, la validation plafonne à 0,70 dès l'epoch 8 — et sa loss repart vers le haut. Le modèle apprend les 1600 images par cœur au lieu d'apprendre ce qui fait un chat.

Pourquoi ?

- 1600 images seulement, c'est très peu
- 1 million de paramètres pour 1600 exemples
- aucune régularisation : rien ne l'empêche de mémoriser

5 Évaluation sur les 400 images de test



0,728

Accuracy

% de bonnes réponses

0,839

Precision

quand il dit « chien », il a raison

0,619

Recall

mais il rate 38 % des chiens

0,712

F1-score

le compromis des deux

Ce que ça dit du modèle

Bien mieux que le hasard (50 %), mais trop prudent sur la classe « chien » : au moindre doute il répond « chat ». D'où une précision haute mais un rappel faible.

6

Amélioration 1 : data augmentation

La meme image, 8 fois, apres data augmentation



L'idée

À chaque epoch, on montre une version légèrement modifiée de chaque image : miroir, rotation, zoom. Le modèle ne voit jamais deux fois la même photo, donc il ne peut plus mémoriser.

Point important du sujet

Les couches d'augmentation de Keras sont actives pendant fit() et automatiquement désactivées au test. L'évaluation n'est donc jamais faussée.

Écart train / validation

0,179 → **0,009**

Accuracy test

0,728 → **0,750**

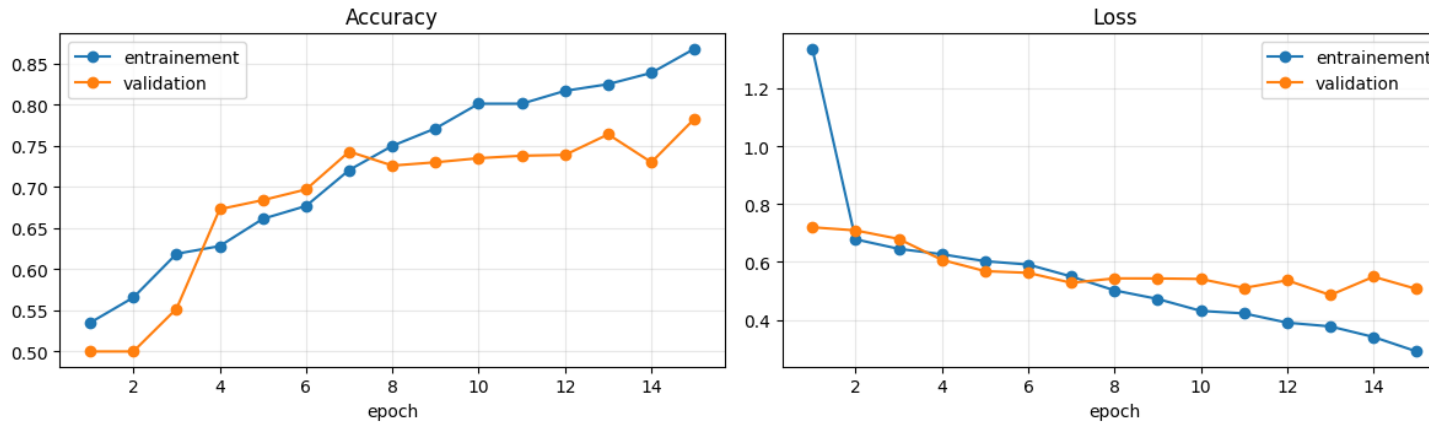
Recall (chiens)

0,619 → **0,858**

7

Amélioration 2 : dropout + batch normalization

Modele C - dropout + batch normalization



Accuracy test

0,780

le meilleur des trois modèles
(contre 0,728 pour le modèle de base)

Dropout (0,5)

À chaque étape, on éteint au hasard la moitié des neurones de la couche dense. Le réseau ne peut plus dépendre d'un neurone en particulier : il est obligé de répartir l'information. Comme l'augmentation, c'est désactivé au moment du test.

Batch normalization

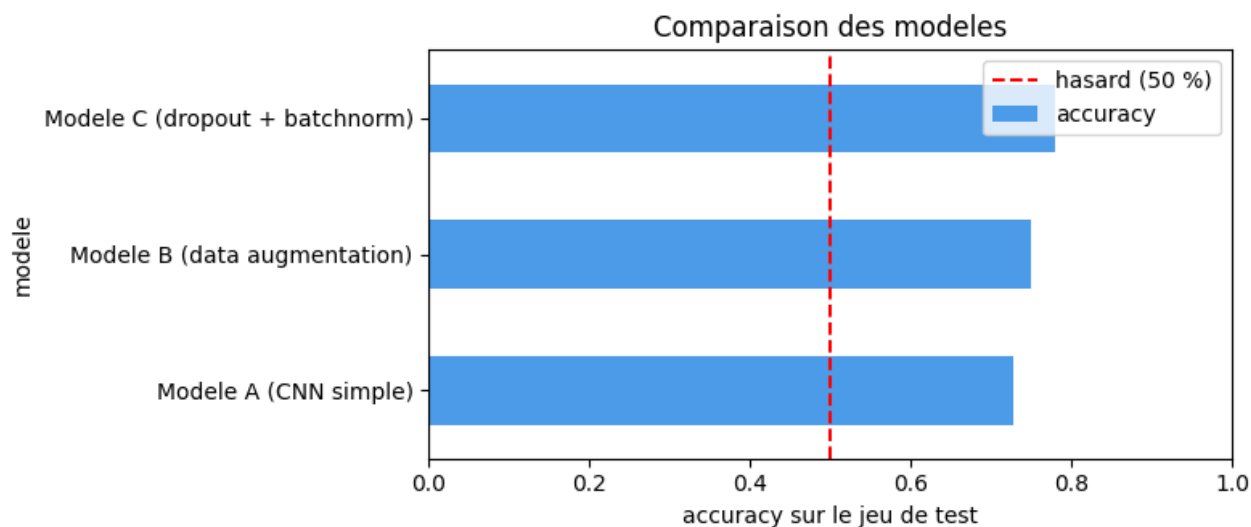
Après chaque convolution, on normalise les sorties sur le batch. L'entraînement est plus stable et plus rapide, avec un léger effet régularisant en bonus.

Piège rencontré : avec le momentum par défaut (0,99) et seulement 16 batchs par epoch, les statistiques de test n'ont pas le temps de converger — la validation reste bloquée à 50 %. Corrigé avec momentum = 0,9.

8

Comparaison des trois modèles

Modèle	Accuracy test	F1-score	Écart train / val	Ce qu'il apporte
A — CNN simple	0,728	0,712	0,179	la base, mais il sur-apprend
B — data augmentation	0,750	0,789	0,009	supprime presque tout l'overfitting
C — dropout + batch norm	0,780	0,810	0,085	le meilleur compromis



Bonus : transfer learning

On réutilise MobileNetV2, déjà entraîné sur 1,4 million d'images ImageNet. On gèle tout le réseau, on remplace juste la dernière couche, et on n'entraîne que celle-là.

**~1300 paramètres à entraîner
au lieu d'un million.**

Ce que je retiens

1

Le vrai problème, c'est le manque de données

1600 images pour des photos naturelles aussi variées, c'est très peu. L'architecture n'était pas le facteur limitant.

2

La régularisation ne rend pas le modèle plus intelligent

elle l'empêche simplement de tricher en mémorisant. C'est ce qui fait passer l'écart de 0,18 à 0,01.

3

Il faut regarder les courbes, pas juste le chiffre final

un détail comme le momentum de la batch norm bloquait la validation à 50 % sans rien casser d'autre.

Pistes suivantes : combiner les deux techniques · fine-tuner MobileNetV2 · EarlyStopping · passer aux 25 000 images du dataset complet

Merci — des questions ?